Project 2: Cache & Memory Performance Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 21, 2026

Abstract

We profile the cache and memory hierarchy of an Intel i9-10900F with four purpose-built microbenchmarks, user-space core counters, memory-controller counters and Intel MLC. Zero-queue pointer-chase latency is 1.80 ns in L1, 4.33 ns for an L2 hit, 18.6 ns in L3 and 103 ns in DRAM with 4 KiB pages (83.4 ns with 2 MiB pages). A loaded-latency probe running beside up to nine streaming cores traces the throughput–latency curve: the knee is at four cores, 30.6 GiB/s (95% of peak) at 1.37× the unloaded latency, matching MLC. With measured latency, Little's Law separates the 55.6 lines the controller serves from up to 152 that queue. Counter-fitted AMAT explains the 37× random-SAXPY slowdown, 2 MiB pages remove a page-walk cost of up to 18.3 ns per access, and memory-controller counters confirm the read-for-ownership cost of writes. A cache-resident buffer reports 32.5 GiB/s of nonexistent "DRAM" bandwidth.

1 Introduction

Every memory reference on this machine is served by one of four levels that differ by two orders of magnitude in latency: a 32 KiB L1d and 256 KiB L2 per core, a 20 MiB shared L3, and DDR4 DRAM. Which level a kernel hits, and how many references it keeps in flight, decides its performance far more than its instruction count. We measure this hierarchy on an Intel i9-10900F with purpose-built microbenchmarks and hardware counters, interpret the results with AMAT and Little's Law, and answer:

1. What is the zero-queue access latency of each level, and how do latency and bandwidth change with access pattern, access size and read/write mix? (Sections 4 to 6)
2. Where does added concurrency stop buying throughput and start costing latency, and does Little's Law hold with measured latency? (Section 7)
3. How do working-set size, cache misses and TLB misses show up in counters and runtime? (Sections 8 to 10)
4. Which benchmark mistake produces misleading numbers? (Section 11)

Every number in the text is generated from the processed data (`data/summary.csv`, Section A); the conclusion (Section 14) summarises the findings.

2 Methodology

2.1 Platform

All measurements were taken on the ten-core i9-10900F desktop (Comet Lake, Skylake-family core, one NUMA node) in Table 1, generated by `scripts/collect_env.sh`; each raw-data directory also stores an `env.txt` snapshot taken just before that experiment.

Table 1: Measurement environment.

CPU	Intel(R) Core(TM) i9-10900F CPU @ 2.80GHz (x86_64)
SIMD ISA	sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c – widest 256-bit (ymm)
Topology	1 socket(s), 10 cores/socket, 1 threads/core (10 of 20 logical CPUs online); NUMA: 1 node(s)
SMT	off
Caches	L1d 320 KiB (10 instances); L2 2.5 MiB (10 instances); L3 20 MiB (1 instance)
Memory	63 GiB total; DIMMs: 2 × 32 GiB DDR4-3200, one per channel (A, B), running at 2933 MT/s (dmidecode)
Pages	base 4096 bytes; THP: always
OS / kernel	Arch Linux; 7.2.9-arch1-1
Frequency policy	governor: performance; driver: intel_pstate; turbo: disabled; EPP: performance
Pinning / isolation	single-thread runs and the loaded-latency probe pinned to CPU 2 (a physical core, not CPU 0); multi-threaded runs place one thread per physical core; no isolcpus
Compilers	gcc (GCC) 16.2.1 20260810; clang version 23.1.3
Tools	perf: perf version 7.2.11-1 (perf_event_paranoid 0); FIO: fio-3.42; MLC: present
GPU (optional)	NVIDIA GeForce RTX 3070; CUDA not available (no CUDA)
Env. snapshot	2026-10-21 20:46 EDT (each raw run directory also stores its own `env.txt`)

2.2 Tools

All benchmarks share the harness in `common/code` (raw monotonic timer, CPU pinning) and a project header `src/memprof.h` (page-policy allocation, ring generators, counters). Raw per-repetition rows are reduced by `tools/summarize` (median, IQR), and `scripts/derive.py` computes every derived number (plateaus, knee, Little's Law, AMAT fit) into `summary.csv`, which the report reads directly.

`mem_latency`	single-thread pointer-chase latency: working-set size, access size, node stride, page size, ring generator, timer, counters
`mem_bandwidth`	streaming bandwidth with T pinned threads: read/write/copy/triad/rwmix, access-size blocks, byte stride
`mem_loaded`	loaded latency: T load-generator cores (stream, rwmix, or k independent chases) plus a concurrent latency probe
`kernel_miss`	SAXPY loop whose footprint, order and page size vary, with counters
`clock_check`	diagnostic: fixed core clock vs. wall time lost by a pinned thread, idle DRAM traffic
Core counters	`perf_event_open(2)` called directly, per thread, user space only [3]
DRAM counters	uncore memory-controller events `data_reads`/`data_writes`, system-wide
Intel MLC	v3.13, run as a normal user (`run.sh mlc`) [1]

We call `perf_event_open(2)` directly (not `perf`, perf version 7.2.11-1) so counts cover only each timed region. Core events (cycles, instructions, L1D read misses, LLC read references and misses, dTLB walks) are counted in user mode for the measuring thread; multiplexed counts are scaled by `time_enabled/time_running`, with the running fraction recorded. DRAM traffic comes from the uncore memory-controller PMU (`uncore_imc data_reads/data_writes`, one count per 64 B line), opened system-wide around the same timed windows (`perf_event_paranoid` ≤ 0); the idle-system background is measured separately (`clock_check`) and quoted next to each result. Intel MLC v3.13 is the external cross-check (Section 3); without root it cannot disable the prefetchers, so it runs with `-e -r`. MLC's MB/s (10^6 B/s) are converted to GiB/s (2^{30} B/s).

2.3 Controls

- **Pinning and isolation:** single-thread runs and the loaded-latency probe pinned to CPU 2 (a physical core, not CPU 0); multi-threaded runs place one thread per physical core. SMT: off. The machine is otherwise idle; all data were collected on 2026-10-21, 20:22–20:39, in one `run.sh all` invocation.
- **Frequency:** governor performance, turbo disabled, so the core runs at its fixed 2.8 GHz base clock (the TSC rate); cycles are ns × 2.80 GHz. User-mode cycles over wall time give a median of 2.77 GHz, and the in-process add-chain probe reads 2.78–2.79 GHz in all 1816 single-thread repetitions (2.69–2.79 GHz in `mem_loaded`). The probe is not used for conversion, because an ACPI event storm once disturbed it; that event (GPE 0x42) was masked, and a pinned thread now keeps 0.997–0.998 of the wall time (Section 12).
- **Pages:** every buffer is a 2 MiB-aligned anonymous mapping whose page size is set by `madvise` *before first touch* (`MADV_HUGEPAGE` or `MADV_NOHUGEPAGE`; the latter is essential because THP is always). Each run records the `AnonHugePages` granted.
- **Warm-up, repetitions, timing:** buffers are pre-faulted and traversed once before timing; 8 repetitions per point (median, IQR bars); each timed interval lasts ≥40 ms to 200 ms, so timer overhead is negligible.
- **Code version and provenance:** the measured sources are byte-identical to commit `b00db66` (verified with `git diff` and file times; the run started from an uncommitted tree on top of `29b8630`). Raw data and per-run metadata are listed in Section A.

3 Validation and Sanity Checks

The microbenchmarks have no reference output to diff, so `scripts/validate.sh` (also `make test`) gates them against physical invariants of any sane hierarchy. `run.sh` executes it immediately before every experiment and stores the output as `validate.txt` with that dataset. The checks are: (i) **monotone levels:** random-chase latency grows strictly, L1 < L2 < L3 < DRAM (Table 2); (ii) **prefetcher:** sequential DRAM "latency" is more than 2× below random, so the prefetcher is engaged for sequential access and defeated by the randomized chase; (iii) **bandwidth ceiling:** L1-resident read bandwidth exceeds the DRAM-resident rate, and ten threads exceed one; (iv) **loaded > unloaded:** the `mem_loaded` probe is slower under four streaming cores than idle; (v) **frequency plausibility:** the add-chain probe lands in 1 GHz to 6 GHz (coarse only; conversions use the fixed clock). Reported but not gating: counters open and return values, and a 2 MiB-page request is granted (`AnonHugePages` > 0) while the 4 KiB policy yields none.

External cross-check: Intel MLC. `run.sh mlc` runs MLC v3.13 (`-e -r`) with its latency thread on CPU 2 and load threads on the other nine cores. Table 2 compares its idle latency with ours and Section 7 its loaded-latency sweep with our knee.

Additional internal checks. Every zero-queue size is measured twice: an LCG ring timed with `CLOCK_MONOTONIC_RAW` and an xorshift64* ring timed with `rdtscp`; on the DRAM plateau they differ by 0.11% (`validate.sh` fails above 15%). The separately written pointer ring of `mem_loaded` gives 103 ns unloaded, against 103 ns from `mem_latency` at 256 MiB. The L1 latency of 5.03 cycles matches this core's published 4–5 cycle load-to-use latency, anchoring the ns→cycle conversion.

Table 2: Zero-queue random-access latency per level (median over plateau sizes of per-size medians of 8 repetitions); cycles = ns × 2.8 GHz. min/max: spread across plateau sizes; IQR%: largest relative IQR at any plateau size. x-check: second method (xorshift64* ring, `rdtscp`), Δ_x its signed difference. 2 MiB: same chase with 2 MiB pages (L2 value from the working-set sweep, Section 8); it removes the page walk (DRAM) and the 4 KiB-page conflict misses (L2). MLC: `--idle_latency -e -r` at 16 KiB, 128 KiB, 4 MiB and 256 MiB (median of 3 runs); negative Δ_{MLC}: MLC is lower. From `data/zero_queue_table.csv`.

Level	Plateau sizes	ns	cycles	min	max	IQR%	x-check	Δ_x %	2 MiB	MLC	Δ_{MLC} %
L1	8–24 KiB	1.80	5.03	1.79	1.80	0.19	1.80	0.0	—	1.40	−22.0
L2	64–192 KiB	5.21	14.6	4.31	6.65	0.21	4.94	−5.3	4.33	5.40	3.6
L3	0.5–15 MiB	18.6	52.1	14.4	24.7	10	18.6	−0.2	—	15.8	−15.1
DRAM	≥80 MiB	103	289	98.8	105	0.27	103	0.1	83.4	75.5	−26.8

4 Zero-Queue Baselines

Hypothesis (from first principles, then compared with the data)

With a single dependent access in flight, latency should rise in steps at the L1 (32 KiB), L2 (256 KiB) and L3 (20 MiB) capacities. We expect roughly 4–5 cycles for L1, ∼12 cycles for L2, a few tens of cycles for L3, and ∼80 ns (hundreds of cycles) for DRAM, matching Skylake-family numbers.

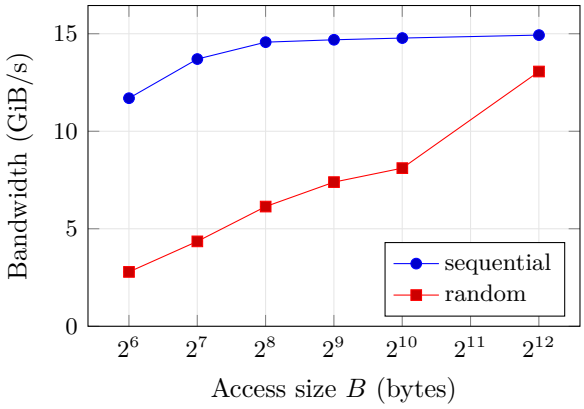
Fixed	one thread pinned to CPU 2; random (Sattolo) pointer chase; one node per 64 B line
Varied	working-set size inside each level's plateau window; page size (DRAM only); method (primary vs. cross-check)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh zero_queue`

The ring is a single random cycle over one node per cache line, so exactly one request is outstanding. *Plateau rule:* a level's latency is not read at its capacity edge, where the set partly spills; `derive.py` takes the median of the per-size medians over all measured sizes s with $2\,C_{prev} \le s \le 0.75\,C_{level}$ ($C_{prev} = 4$ KiB for L1); DRAM uses every size $\ge 4\times$ the L3.

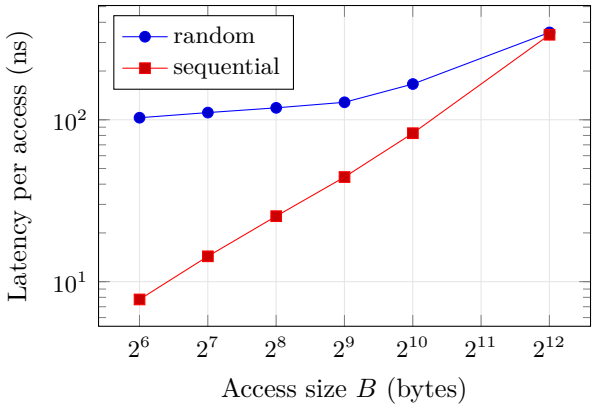
4.1 Analysis

Latency is 1.80 ns (5.03 cycles) in L1. The L2 plateau with 4 KiB pages is 5.21 ns (14.6 cycles), the median of sizes that measure 4.31–6.65 ns. This is *not* the L2 hit latency: at 128 KiB and 192 KiB, 4 KiB pages already cause L2 conflict misses (Section 8). With 2 MiB pages the same sizes give the L2 hit latency, 4.33 ns (about 12 cycles). L3 latency spans 14.4–24.7 ns (median 18.6 ns), because larger sets also miss the second-level TLB. Latency reaches 103 ns (289 cycles) in DRAM. With 2 MiB pages the DRAM figure drops to 83.4 ns; the 19.6 ns difference is the page walk that every random 4 KiB access beyond the TLB reach pays (Section 10). Our second method agrees within 5.3% at every level (largest on the L2 plateau, 5.3%; DRAM 0.11%). L1, L2 and L3 match the hypothesis; DRAM with 4 KiB pages is slower than the ∼80 ns we expected, and the 2 MiB-page value shows the excess is mostly the page walk.

Comparison with Intel MLC. MLC agrees with our 4 KiB-page L2 plateau (5.40 ns, 3.6% relative to ours) but is lower at the other levels: L1 1.40 ns (−22%), L3 15.8 ns (−15%), and DRAM 75.5 ns (−27%). In L1, MLC's 4 clocks are a plain pointer dereference; our index ring's scaled-index



(a) Streaming bandwidth of B-byte blocks.



(b) Latency of one dependent B-byte access.

Figure 1: Access pattern and access size, DRAM-resident, one core. Strided rows of `pattern_bw.csv` ($B = 0$) are dropped from the log axis and shown in Figure 2.

address costs one extra cycle. In DRAM, MLC returns the same 75.5 ns with and without `-L` (2 MiB pages): its default `-D` random range of 4 KiB keeps consecutive accesses inside one page, so it takes no page walks. Even against our 2 MiB-page value (83.4 ns) it is 9.5% lower. MLC thus measures a best-case local latency; we keep ours because AMAT and TLB analysis need fully random access.

5 Access Pattern and Granularity

Hypothesis (from first principles, then compared with the data)

Access size. Reading a whole B-byte block per access should leave sequential bandwidth roughly unchanged but raise random bandwidth steeply with B, because a larger block amortises one DRAM latency over more lines and lets the prefetchers help. The latency of one dependent B-byte access should grow far less than B, since its lines are fetched in parallel. *Stride.* With one 8 B element used per stride, useful bandwidth should fall roughly as 1/stride once the stride reaches a 64 B line.

Fixed	1 thread pinned to CPU 2; 256 MiB DRAM-resident region; 100% reads; 4 KiB pages
Varied	pattern (sequential, random); access size $B = 64 \ldots 4096$ B (primary); byte stride 8–1024 (secondary)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh pattern`

Definitions. *Access size B*: every byte of a B-byte block is read (256 bit loads) before the next block, visited in address or shuffled order. For latency, the next block's address depends on all lines of the current one, so one access is one dependent B-byte request. *Stride* (Figure 2): one 8 B scalar element used per stride.

5.1 Analysis: access size

At $B = 64$ B sequential blocks stream at 11.7 GiB/s and random blocks at 2.79 GiB/s (4.19× apart). Growing the block to 256 B and 1024 B lifts random bandwidth to 6.14 GiB/s and 8.11 GiB/s (sequential: 14.6 GiB/s, 14.8 GiB/s), shrinking the gap to 1.82×; at 4096 B random blocks reach 13.1 GiB/s, close to the sequential 14.9 GiB/s. Contiguous lines within a block overlap their misses

and engage the prefetchers; sequential bandwidth is flat from 256 B up. Per request, a random dependent 64 B access costs 103 ns but a 1024 B one only 166 ns—16 lines for far less than 16 latencies—so one dependent chain moves 5.74 GiB/s instead of 0.579 GiB/s. Sequentially chained blocks cost 7.77 ns (64 B) and 82.6 ns (1024 B), the gap to random being latency the prefetcher hides; at 4096 B it is gone (335 ns vs. 346 ns random): every block starts a new 4 KiB page, and the prefetcher does not cross pages.

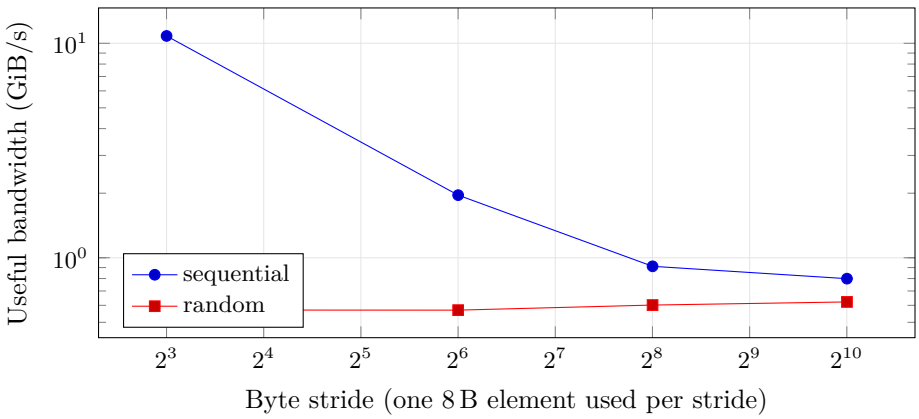
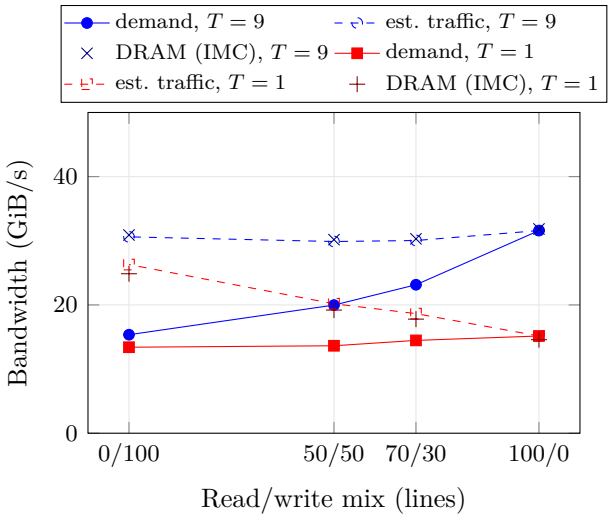


Figure 2: Secondary result: useful bandwidth vs. byte stride with scalar 8 B loads.

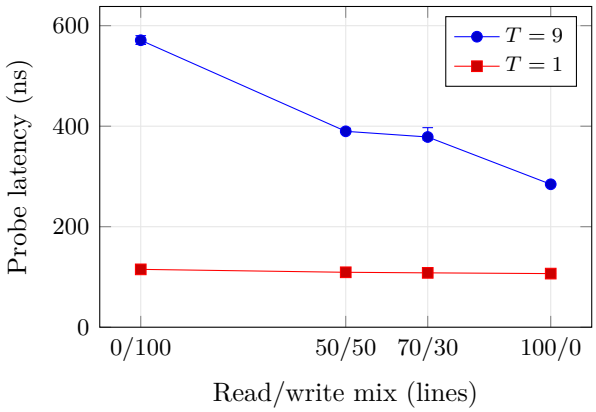
5.2 Analysis: stride, and the line-traffic anomaly

Sequential useful bandwidth is 10.8 GiB/s at unit (8 B) stride and 1.96 GiB/s at 64 B. At 64 B stride every access pulls a whole line, so *line* traffic is 8× the useful bandwidth: 15.7 GiB/s, i.e. 1.45× the unit-stride rate. So the obvious unit-stride scalar read does *not* measure the single-core streaming limit: with eight loads and adds per line, fewer lines fit in the out-of-order window. Full-line vector reads of large blocks reach 14.9 GiB/s (at 64 B blocks, with loop overhead, only 1.08× the scalar kernel: 11.7 GiB/s). The single-core streaming limit is therefore about 15 GiB/s to 16 GiB/s; the loaded-latency study (Section 7) uses a two-loads-per-line vector stream.

Contrary to our hypothesis, useful bandwidth does *not* keep falling as 1/stride beyond one line: it is 0.912 GiB/s at 256 B and 0.799 GiB/s at 1024 B, so touched-line traffic drops from 15.7 GiB/s to 7.30 GiB/s and 6.39 GiB/s. Each access is then a separate miss, so misses in flight, not bandwidth, limit it. At 64 B stride DRAM reads are 15.1 GiB/s, 0.961× the touched-line traffic. At 256 B and 1024 B strides DRAM reads are 2.03× and 2.04× the touched lines (14.8 GiB/s and 13.0 GiB/s): the spatial (adjacent-line) prefetcher fetches the other half of each 128 B pair, so every useful 8 B costs two lines of DRAM traffic. Random 64 B blocks show the same pairing (5.49 GiB/s of DRAM reads for 2.79 GiB/s useful). Random strided access is flat at 0.571–0.622 GiB/s: every access is an independent miss.



(a) Throughput.



(b) Loaded latency of a random read.

Figure 3: Read/write mixes: throughput and concurrently measured latency. Demand: bytes the program reads and writes; est. traffic: adds one RFO line per written line; DRAM (IMC): memory-controller read+write count over the same windows. The unloaded probe latency is 103 ns.

6 Read/Write Mix

Hypothesis (from first principles, then compared with the data)

Every written line needs a read-for-ownership (RFO) and later a write-back, so it costs about two lines of DRAM traffic plus bus turnarounds. Demand bandwidth should therefore fall as the write fraction grows while DRAM traffic stays nearly constant, and loaded latency should be highest for write-heavy mixes, whose write-backs compete with the probe's reads.

Fixed	128 MiB per generator core; sequential; ordinary (write-allocate) full-line stores; 4 KiB pages
Varied	reads per 10 *lines*: 10 (100/0), 7 (70/30), 5 (50/50), 0 (0/100); $T = 1$ and 9 generator cores
Latency	concurrent random pointer-chase probe on CPU 2 (`mem_loaded`); $T = 0$ gives the unloaded value
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh rw_mix`

Mix definition. Of every 10 consecutive 64 B lines, the first R are read and the remaining $10 - R$ are fully overwritten, so the ratio holds for the traffic the memory system sees. Mixing 8 B elements instead is a trap: every source line is read *and* every destination line written, so a nominal "70/30" moves about three lines per useful line (Section 13).

6.1 Analysis

With nine generator cores, demand bandwidth is 31.6 GiB/s for 100% reads, 23.1 GiB/s at 70/30, 20.0 GiB/s at 50/50 and 15.4 GiB/s for 100% writes, falling with the write fraction as predicted. The memory-controller counters measure 31.8 GiB/s to 30.9 GiB/s across the mixes, within about 1% of the RFO-counted estimate (31.6 GiB/s to 30.6 GiB/s; idle background 0.088 GiB/s); for pure writes the controller reads 15.6 GiB/s (the RFOs) and writes 15.3 GiB/s. Total DRAM traffic stays at about 30 GiB/s whatever the mix, as does MLC's peak-injection bandwidth (31.8 GiB/s all reads, 30.0 GiB/s at 3:1, 30.6 GiB/s at 1:1). The cost of writes is the RFO, not a loss of controller efficiency.

Probe latency rises from 284 ns (reads) to 571 ns (writes): at the same total DRAM traffic, pure writes double it (2.01×) and the 70/30 and 50/50 mixes raise it by about a third (1.33×, 1.37×): write-backs and RFOs share the probe's queues, and bus turnarounds lengthen every read's wait.

One generator core ($T = 1$) is not controller-bound: demand bandwidth falls only from 15.2 GiB/s to 13.4 GiB/s (perhaps because stores retire without waiting for their RFO; untested), while DRAM traffic grows from 14.6 GiB/s to 24.9 GiB/s. Here the counters read 0.945–0.962× the RFO-counted estimate, even for pure reads, a gap we did not resolve; *untested hypothesis:* the estimate counts lines still in the L3 from the previous pass, which a buffer many times the L3 would test. The probe barely notices one core (107–115 ns). On all ten cores, `copy` delivers 20.0 GiB/s and `triad` 21.9 GiB/s; copy reads 20.1 GiB/s and writes 10.0 GiB/s (2:1, source plus RFO), and triad reads 22.1 GiB/s and writes 7.33 GiB/s (3:1). Non-temporal stores (no RFO) were out of scope.

7 Intensity Sweep and the Throughput–Latency Knee

Hypothesis (from first principles, then compared with the data)

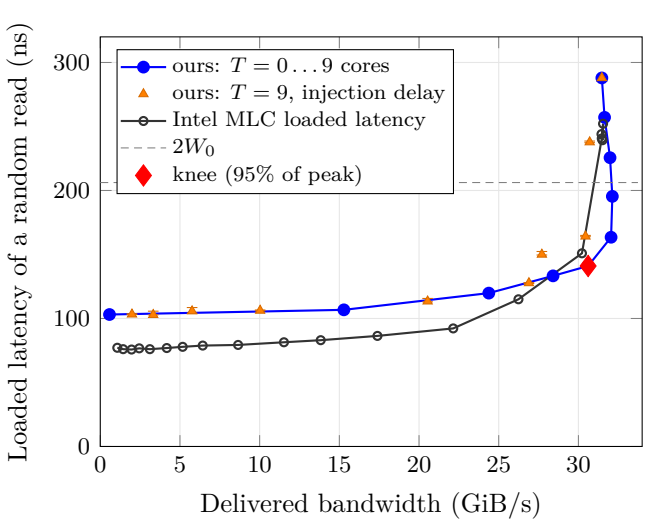
One core can keep only about a dozen demand misses in flight (its L1 fill buffers), so it cannot saturate DRAM. Adding load should raise bandwidth until the controller saturates in the low tens of GiB/s (two DDR4 channels); past that, extra load should buy almost no bandwidth but sharply raise latency (queueing). By Little's Law, saturation needs tens of outstanding lines—more than one core provides.

Generators	`mem_loaded`: $T = 0 \ldots 9$ cores (all physical cores except CPU 2), each streaming its own 128 MiB region with 256 bit loads; then $T = 9$ throttled by an injection delay of 8–2048 empty iterations per line
Probe	random pointer chase over 256 MiB on CPU 2, measured in the same window as the generators' throughput
Known MLP	T cores each running k independent random chases: $T = 1$, $k = 1 \ldots 32$; $T = 2 \ldots 9$, $k = 8$
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh intensity`

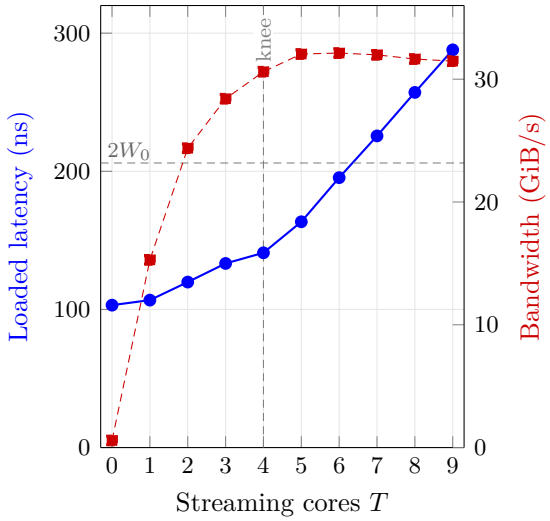
Method and knee criterion. Each point reports *both* axes from one window: generator progress counters are snapshotted around a 200 ms probe run. Throughput is all lines moved per second; latency is the probe's time per dependent demand read *under* that load ($T = 0$: unloaded $W_0 = 103$ ns). The knee is the first point, in order of offered load, at which bandwidth reaches 95% of the peak; maximum throughput/latency ("power") is a second criterion and the point where latency reaches $2W_0$ a latency reference, all computed by `derive.py`.

7.1 Knee and diminishing returns

The curve has the textbook shape (Figure 4, Table 3). The knee is at $T = 4$: bandwidth has climbed to 30.6 GiB/s, 95% of the 32.1 GiB/s peak, while latency has risen only from 103 ns to 141 ns (1.37×). The throughput/latency maximum falls on the same point ($T = 4$, 30.6 GiB/s, 141 ns), so both criteria agree. Every core added after the knee buys almost no bandwidth (peak at $T = 6$; $T = 9$ still 31.5 GiB/s), while latency climbs to 288 ns. Latency crosses $2W_0$ only at 32.1 GiB/s, between $T = 6$ and $T = 7$, at the bandwidth peak itself (injection-delay sweep: 30.6 GiB/s). Memory-controller counters confirm the throughput axis (1.01× and 1.01× the delivered bandwidth at $T = 4$ and $T = 9$). Beyond the knee requests no longer finish faster; they queue. Against the theoretical peak of two DDR4 channels at 2933 MT/s (`dmidecode`), $2 \times 8\,\text{B} \times 2933\,\text{MT/s} = 46.9\,\text{GB/s} = 43.7\,\text{GiB/s}$,



(a) Latency vs. delivered bandwidth.



(b) Latency (left, blue) and bandwidth (right, red) vs. cores.

Figure 4: Measured throughput–latency curve. (a) Our load levels with Intel MLC's loaded-latency sweep. (b) The thread sweep: bandwidth flattens at the knee while latency keeps rising. Both coordinates are medians of 8 windows (bars: IQR of latency); the dashed line marks $2W_0$.

the measured peak (32.1 GiB/s) is 73 % and the knee 70 %, the usual practical limit for DDR4 (refresh, bank conflicts, bus turnarounds).

Comparison with Intel MLC. MLC's `--loaded_latency` sweep starts lower, at 77.1 ns (Section 4). By the same 95% rule, its knee is at 30.2 GiB/s and 151 ns (peak 31.5 GiB/s, injection delay 50); its latency doubles at 30.3 GiB/s and reaches 244 ns at full load. The instruments agree on the ceiling (within 2%) and the shape.

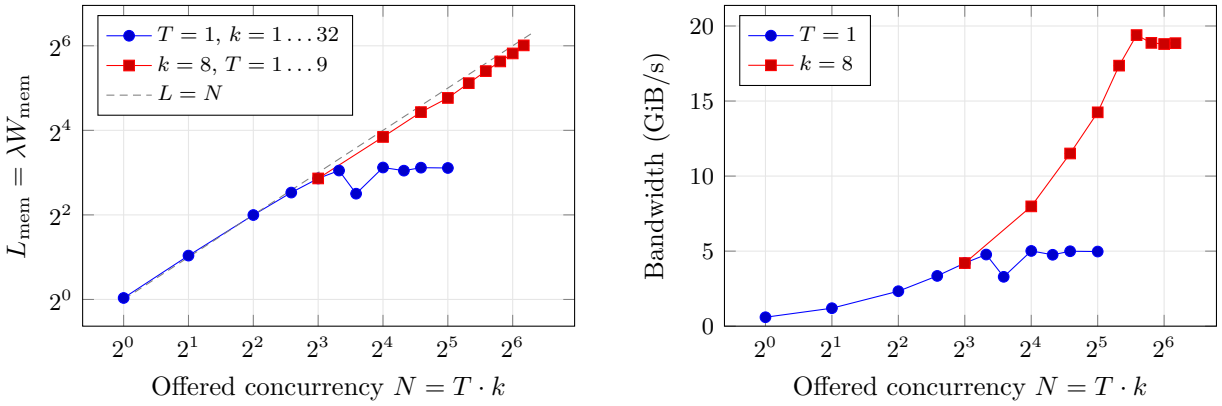
7.2 Little's Law with measured latency

For every load level, $L = \lambda W$ [2] is the mean number of lines in flight, with λ the delivered line rate and W the *measured* loaded latency of the same window; at the knee $L = 72.4$. The usual shortcut $L_0 = \lambda W_0$ rises to 55.6 lines and then stays flat: the concurrency the memory system can *serve*. L keeps growing to 152 lines at $T = 9$; the difference is requests waiting in queues, which is why latency rises with no bandwidth gain.

What the known-concurrency sweep shows. With W taken as each chain's own step time, L/N is 1.00–1.01, but that holds by construction and only confirms the progress accounting. The informative quantity is $L_{\text{mem}} = \lambda W_{\text{mem}}$, with W_{mem} the probe's concurrent memory latency (103 ns with one generator core). One core peaks at 5.01 GiB/s ($k = 16$) and its L_{mem} never exceeds 8.70 lines (Figure 5); extra chains wait inside the core, so step time grows instead (384 ns at $k = 32$). This is consistent with the core's 12 L1D fill buffers. (We use the envelope; $k = 12$ is an unresolved outlier, Section 12.) With $k = 8$ per core, adding cores raises bandwidth to 19.4 GiB/s at $T = 6$ ($L_{\text{mem}} = 42.2$, $W_{\text{mem}} = 130$ ns). Random demand reads thus reach only about 60% of the streaming peak: nine such cores keep about seven lines each in flight, and the memory system queues long before it is fully used.

Table 3: Little's Law across the thread sweep (no injection delay), generated from `data/littles_law.csv`. λ = delivered lines/s (64 B lines), W = loaded probe latency, $L = \lambda W$; $L_0 = \lambda W_0$ with the unloaded W_0.

T	BW (GiB/s)	λ (Mlines/s)	W (ns)	$L = \lambda W$	$L_0 = \lambda W_0$
0	0.58	10	103.1	1.0	1.0
1	15.29	256	106.7	27.4	26.4
2	24.38	409	119.8	49.0	42.2
3	28.41	477	133.3	63.5	49.1
4	30.61	514	141.0	72.4	52.9
5	32.05	538	163.5	87.9	55.4
6	32.13	539	195.4	105.3	55.6
7	31.98	537	225.6	121.1	55.3
8	31.64	531	257.1	136.5	54.7
9	31.47	528	287.9	152.0	54.4



(a) Lines in flight in memory vs. offered N.

(b) Random-read bandwidth vs. N.

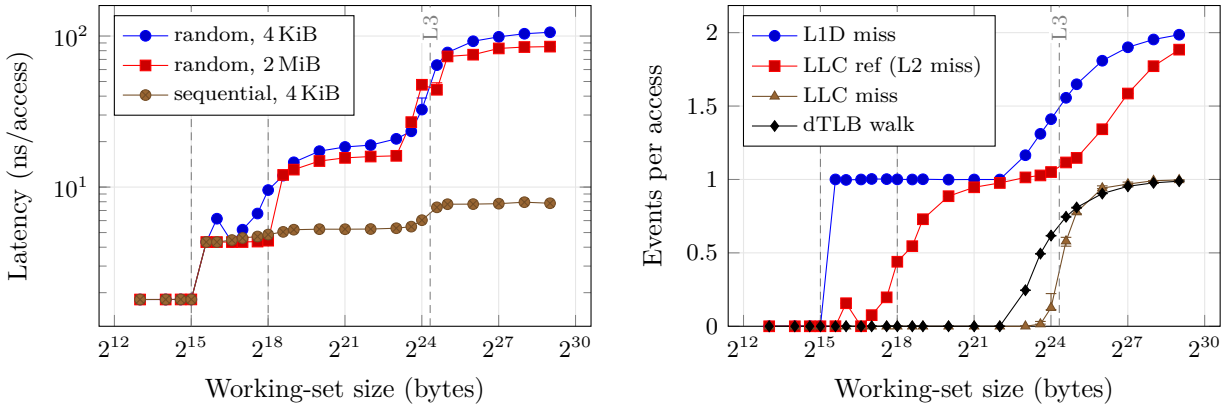
Figure 5: Known-concurrency sweep: k independent random chases per core (median of 8 windows, bars: IQR). L_{mem} uses the probe's concurrent memory latency W_{mem}; the dashed line $L = N$ would mean every chain always had a request in memory.

Streaming vs. demand concurrency. One streaming core keeps $L = 27.4$ lines in flight at 15.3 GiB/s (Table 3), about three times its 8.70 demand misses, because most streamed lines are fetched by the L2 prefetcher, whose requests are tracked in the L2's queue rather than the fill buffers. The 55.6 lines the controller serves at saturation are about two streaming cores' worth of concurrency (four with queueing), but six random-access cores' worth.

8 Working-Set Sweep

Hypothesis (from first principles, then compared with the data)

Chase latency should step up at each capacity (32 KiB, 256 KiB, 20 MiB), and the counters should show why: L1D misses per access rising from 0 to 1 past 32 KiB, LLC misses rising to 1 past 20 MiB, and dTLB walks appearing past the ~6 MiB 4 KiB-page TLB reach, but not with 2 MiB pages.



(a) Latency.

(b) Counters, random chase, 4 KiB pages.

Figure 6: Working-set sweep. Left: latency with cache capacities marked. Right: user-space counters per access for the random 4 KiB-page chase; each step in latency coincides with a miss ratio rising towards one.

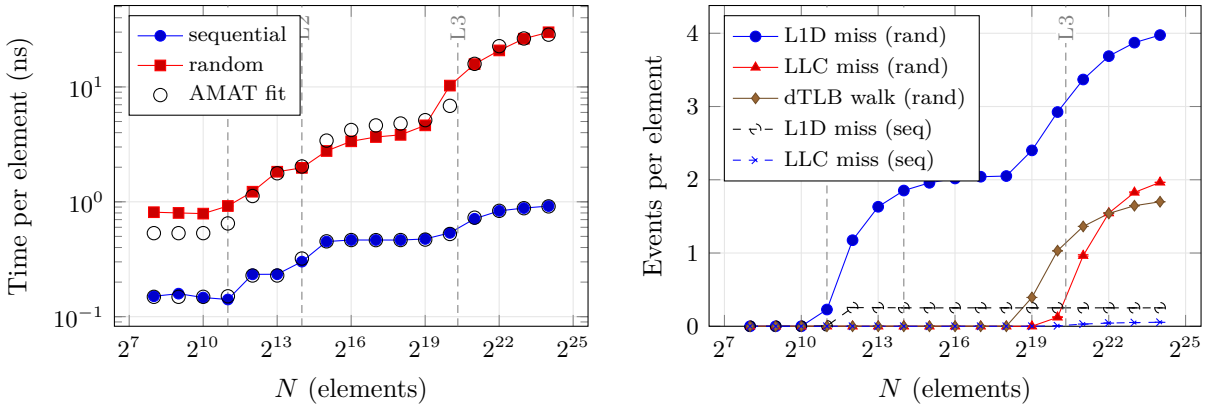
Fixed	1 thread pinned to CPU 2; one node per 64 B line; user-space counters
Varied	working-set size 8 KiB–512 MiB; random vs. sequential; 4 KiB vs. 2 MiB pages (random)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh working_set`

8.1 Analysis

The latency staircase and the counters line up level by level (Figure 6). L1D misses per access jump from ≈0 to 1 between 32 KiB and 48 KiB, exactly at the L1 capacity. Page walks start at 8 MiB, just past the ≈6 MiB STLB reach, which is why the 4 KiB latency already climbs inside the L3 region (20.9 ns vs. 16.1 ns with 2 MiB pages). LLC misses appear from 12 MiB to 16 MiB, before the nominal 20 MiB, and reach one per access by 64 MiB. In DRAM the random 4 KiB-page chase incurs 0.988 page walks and 1.99 L1D misses per access (1.00 with 2 MiB pages): the walker's reads miss the L1 too, so the 4 KiB curve sits above the 2 MiB curve (106 ns vs. 85.1 ns at 512 MiB). The sequential chase costs only 7.82 ns (0.0686 demand LLC misses per access vs. 1.000): the prefetcher turns its misses into hits.

L2 conflict misses with 4 KiB pages. L2 misses (LLC references) already appear inside the L2 window, non-monotonically: 0.16 per access at 64 KiB, 0.00017 at 96 KiB, 0.076 at 128 KiB and 0.20 at 192 KiB, approaching one only by 1 MiB to 2 MiB. At 192 KiB the 4 KiB-page chase costs 6.68 ns (0.197 L2 misses per access), the 2 MiB-page chase 4.36 ns (0.00248). The likely cause is page placement: 4 KiB pages land in arbitrary frames, so some L2 sets receive more lines than their associativity, while a contiguous 2 MiB page spreads lines evenly. Placement is decided per allocation, so a fresh buffer can reverse the order (6.18 ns and 5.22 ns at 64 KiB and 128 KiB here, 4.31 ns and 6.10 ns in the zero-queue run).

A page-size crossover at the L3 edge. At 12 MiB and 16 MiB the 2 MiB-page chase is *slower* (26.9 ns vs. 23.4 ns; 47.6 ns vs. 32.6 ns), with more LLC misses (0.15 vs. 0.015; 0.46 vs. 0.13) and no page walks; from 24 MiB the order reverts (44.1 ns vs. 64.2 ns). So page tables competing for the L3 cannot explain the early LLC misses. *Untested hypothesis:* a 12 MiB to 16 MiB buffer occupies only 6–8 contiguous 2 MiB frames, so any uneven mapping onto the hashed L3 slices and sets is



(a) Runtime and fitted AMAT model.

(b) Counters per element.

Figure 7: SAXPY time per element and counters vs. footprint and traversal order (two `double` arrays, $16N$ bytes; lines mark where $16N$ crosses L1/L2/L3; random order also reads a $4N$-byte index array, so its $20N$-byte footprint crosses each capacity at $0.8\times$ the marked N). Open circles: AMAT model fitted to the counters (Equation (1)).

shared by every line, whereas 4 KiB pages average it out; repeating these points over fresh huge-page allocations would test it. Some counters were multiplexed (minimum running fraction 0.662) and are scaled.

9 Cache-Miss Impact

Hypothesis (from first principles, then compared with the data)

As the SAXPY footprint grows past each cache, time per element should rise with the misses per element. Sequential order should rise gently (the prefetcher turns misses into hits), random order sharply. Because independent iterations overlap their misses, the effective DRAM miss cost should be well below the zero-queue latency.

Kernel	$y_i = a\,x_i + y_i$ (float64), two arrays, 4 KiB pages
Varied	$N = 2^8 \ldots 2^{24}$ (footprint $16N$ bytes sequential; $20N$ random, which adds a 4-byte index per element); order (sequential, random permutation)
Counters	L1D read misses, LLC read references/misses, dTLB walks, cycles, instructions (user space, per element)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh cache_miss`

9.1 Analysis

Sequential SAXPY rises from 0.151 ns per element in L1 to 0.918 ns in DRAM, a 6.1× span, with only 0.0549 demand LLC misses per element at the largest size. Random traversal rises from 0.812 ns to 29.9 ns, a 37× span, with 3.98 L1D misses, 1.96 LLC misses and 1.70 page walks per element at $N = 2^{24}$ (a line of x, a line of y and page-walk reads). At DRAM sizes the random curve is 33× the sequential one: that gap is the miss cost the prefetcher hides.

9.2 AMAT fitted to measured miss counts

Classic AMAT, $t_{L1} + m_{L1}(t_{L2} + m_{L2}(t_{L3} + m_{L3}\, t_{\mathrm{DRAM}}))$, assumes serialised misses. We instead fit its per-element form to the counters, separately for each order:

$$t_{\mathrm{elem}} = t_0 + a_2\, h_{L2} + a_3\, h_{L3} + a_D\, m_D, \tag{1}$$

with per-element hits served by L2 (h_{L2} = L1D misses − LLC references), by L3 (h_{L3} = LLC references − LLC misses) and DRAM misses (m_D = LLC misses); t_0 absorbs L1-hit work. Least squares over the 17 footprints gives, for random order, $t_0 = 0.533$ ns, $a_2 = 0.502$ ns, $a_3 = 2.12$ ns, $a_D = 12.3$ ns ($R^2 = 0.986$); for sequential order $a_D = 8.37$ ns ($R^2 = 0.999$). The fitted a_D is the *exposed* cost of one DRAM miss; the zero-queue DRAM latency divided by it gives the average number of overlapped misses: 8.3 for random and 12 for sequential SAXPY (whose remaining misses come in long, predictable runs). For random order the zero-queue L2 and L3 latencies are likewise 10× and 8.8× the fitted a_2 and a_3. All three factors are close to the 8.70 demand misses one core keeps in flight (Section 7). Classic serialised AMAT confirms this at $N = 2^{24}$: with the Table 2 latencies, the measured $h_{L2} = 0.33$, $h_{L3} = 1.68$, $m_D = 1.96$ and the fitted t_0, it predicts 236 ns per element (202 ns of it DRAM) against the measured 29.9 ns, a 7.9-fold overprediction.

Caveats: the L2/L3 terms are only roughly determined. The largest relative errors (random up to 34%, sequential 6.6%) are at $N = 2^8$ (34%, loop overhead) and $N = 2^{20}$ (33%), where the footprint equals the L3: only 0.123 LLC misses but already 1.03 page walks per element, and the model has no term for walks without DRAM misses (2 MiB pages: 5.42 ns, Table 4).

10 TLB-Miss Impact

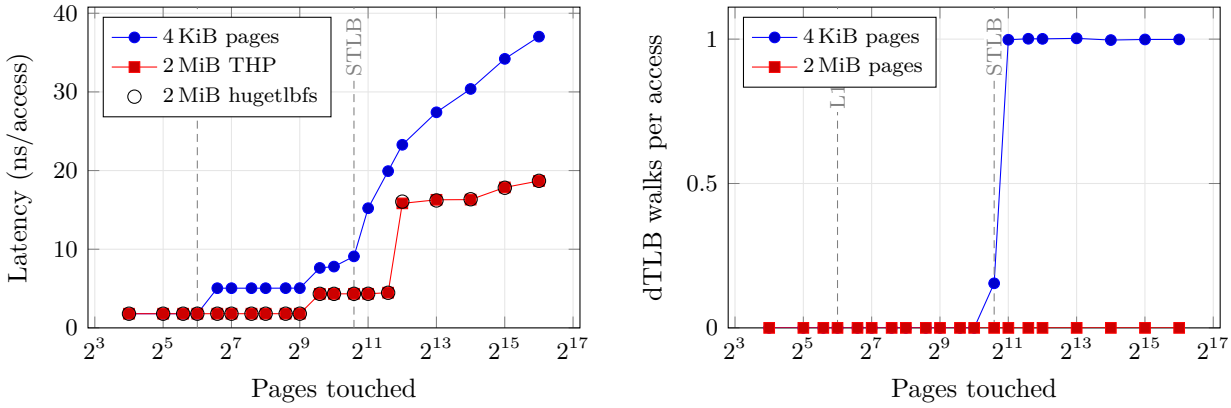
Hypothesis (from first principles, then compared with the data)

With 4 KiB pages, a dependent access to a random page should cost extra once the pages touched exceed the L1 dTLB (64 entries), and again past the second-level TLB (1536 entries, ≈6 MiB), where every access walks. With 2 MiB pages (512× the reach per entry) the walks and extra latency should vanish.

Chase	one node per 4 KiB page (node stride 4160 B = page + one line, so nodes fall into different cache sets); random order; 16–65536 pages
SAXPY	random-order $y_i = a\, x_i + y_i$, $N = 2^{18} \ldots 2^{24}$
Varied	page size: 4 KiB (`MADV_NOHUGEPAGE`) vs. 2 MiB transparent huge pages (`MADV_HUGEPAGE`), both set before first touch; the chase also uses reserved `hugetlbfs` 2 MiB pages (`MAP_HUGETLB`)
Verification	`AnonHugePages` per run; dTLB-walk counter per access
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh tlb`

10.1 Analysis

Page size is the only difference between the curves, so any gap is TLB cost; both TLB levels show as separate cliffs (Figure 8). Up to 64 pages both page sizes cost the same (1.80 ns vs. 1.80 ns). At 128 pages the 4 KiB chase jumps to 5.05 ns while the 2 MiB chase stays at 1.80 ns, with no walks in either: the 3.25 ns (9.10 cycles) gap is an L1 dTLB miss that hits the STLB. Near the STLB capacity walks begin (0.154 per access at 1536 pages), and at 2048 pages every 4 KiB access walks (0.998 walks per access): 15.2 ns against 4.33 ns, a 10.9 ns (30.4-cycle) page-walk penalty while page-table entries still hit in cache. Past 4096 pages the data footprint leaves L2, raising both curves (huge



(a) Page-stride chase latency.

(b) Completed page walks (counter).

Figure 8: TLB reach: dependent access to one line per page. The gap between the curves is the cost of TLB misses; the counter confirms walks occur only with 4 KiB pages past the STLB reach.

Table 4: Random-order SAXPY with 4 KiB vs. 2 MiB pages (median of 8 repetitions; footprint $20N$ bytes). Per element: t ns, w page walks, m LLC misses; subscripts give the page size; speedup $= t_{4K}/t_{2M}$. From `data/tlb_saxpy_table.csv`.

N	MiB	t_{4K}	t_{2M}	w_{4K}	w_{2M}	m_{4K}	m_{2M}	speedup
2^{18}	5	3.83	3.79	0.00	0.00	0.00	0.00	1.01
2^{19}	10	4.64	3.86	0.39	0.00	0.00	0.00	1.20
2^{20}	20	10.3	5.42	1.03	0.00	0.12	0.04	1.89
2^{21}	40	15.8	10.6	1.36	0.00	0.98	0.96	1.48
2^{22}	80	21	14.2	1.54	0.00	1.55	1.54	1.48
2^{23}	160	26.5	16.1	1.65	0.00	1.83	1.81	1.65
2^{24}	320	30	16.9	1.70	0.00	1.96	1.94	1.77

pages: 15.8 ns), so only the gap is attributed to the TLB. It grows as the page tables outgrow the caches: 14.1 ns at 16384 pages and 18.3 ns at 65536 pages (37.0 ns vs. 18.7 ns, 1.98×), while 2 MiB pages stay at 0 walks.

For random SAXPY (Table 4) at $N = 2^{24}$, 2 MiB pages cut page walks from 1.70 to 0 per element and runtime from 30.0 ns (`cache_miss` run: 29.9 ns) to 16.9 ns per element, a 1.78× speedup, a little below the 1.98× of the dependent chase: overlap hides only a small part of the walks, since each adds dependent page-table reads before its data miss. The largest speedup is at $N = 2^{20}$, where the 2 MiB-page data still mostly hit in the L3 but 4 KiB pages already walk about once per element.

Huge pages are verified, not assumed. Every "huge" row was fully backed by `AnonHugePages` (e.g. 266 240 KiB for the 65536-page chase, 327 680 KiB for SAXPY at $N = 2^{24}$; summarised in `anon_thp.txt`), every 4 KiB row records 0 KiB, and the walk counter confirms it. Reserved `hugetlbfs` pages (`MAP_HUGETLB`) give the same latency as THP (16.0 ns vs. 15.8 ns at 4096 pages, 18.7 ns vs. 18.7 ns at 65536), with zero walks.

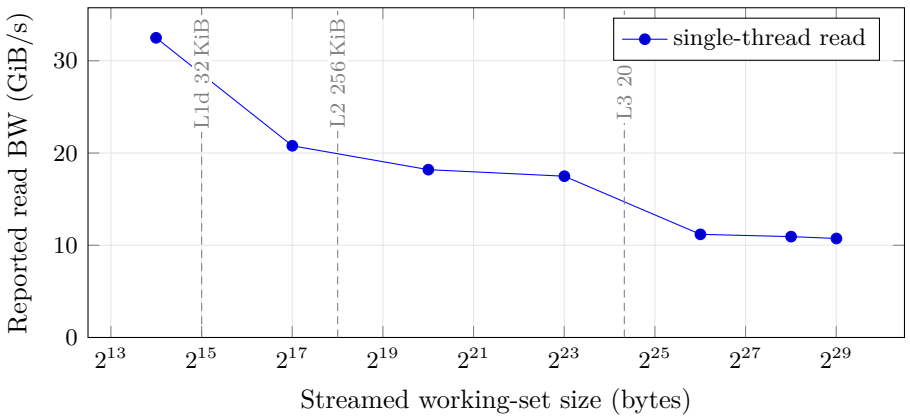


Figure 9: The same streaming kernel "measures" wildly different bandwidth depending only on the buffer size. A 16 KiB buffer reports 32.5 GiB/s; the honest DRAM-resident figure is 10.7–11.2 GiB/s. Nothing about the code changed—only where the data lived.

11 Benchmark Pitfall

Hypothesis (from first principles, then compared with the data)

A "DRAM bandwidth" benchmark that re-streams a cache-sized buffer will report several times the real memory bandwidth, because every pass after the first hits in cache. We expect an L1/L2-resident buffer to report tens of GiB/s and a DRAM-resident one to settle near the single-core streaming figure of Section 5.

Pitfall	Measuring cache reuse instead of DRAM: picking a buffer that fits in cache
Wrong setup	Working set 16 KiB (16 320 B after rounding to whole loop units; L1-resident), re-streamed many times, one thread
Right setup	Working set 64 MiB to 512 MiB (well past the 20 MiB LLC)
Script	`Cache_and_Memory_Performance_Profiling/scripts/run.sh pitfall`

11.1 Explanation

At 16 KiB the kernel reports 32.5 GiB/s, falling as the set spills L1 and L2; once the set is 64 MiB or larger—genuinely DRAM-resident—it settles at 10.7–11.2 GiB/s, a 3.0× overstatement. (This is the scalar unit-stride read of Section 5, so its DRAM figure is not the single-core peak.) The small-buffer number is L1/L2 bandwidth from a loop that re-reads the same cached lines. An AI-generated bandwidth microbenchmark easily falls into this trap, because a 16 KiB array is a natural default and tens of GiB/s look plausible. Two cheap detectors flag it: (1) the result *changes with buffer size* beyond the LLC, and (2) one core reports as much as the whole memory system delivers to *all* cores (32.1 GiB/s, Section 7). We avoid the pitfall by sizing every point against the cache capacities and using DRAM working sets at least 3× the LLC.

12 Anomalies and Limitations

12.1 Unexpected results and how they were resolved

- **An ACPI event storm stole wall time from every core.** In a first run (data discarded), the add-chain probe read either 2.79 GHz or about 2.41 GHz with turbo off. Hypothesis: the clock is constant but the thread loses wall time. In the retained `clock_check` run the core clock was 2.79 GHz in all 80 windows, but in the 39% with an ACPI event burst (about 114 per window) the pinned thread ran user code for only 0.866 of the wall time (quiet: 0.998), with nothing charged by the OS—consistent with firmware (SMM) handling. The source, GPE 0x42, was masked and all data re-measured (Section 2).
- **Smaller items.** The add-chain probe was code-alignment sensitive (now a 32-byte-aligned loop of eight ADDs). The unit-stride scalar read is not the single-core peak (Section 5). With one generator core, $k = 12$ chains deliver less bandwidth than $k = 10$ or $k = 16$ at the same memory latency, a generator-loop artifact (unresolved; envelope used). The two unloaded DRAM latencies, 6% apart in the disturbed run, now agree (103 ns, 103 ns).
- **Methodology traps caught early:** $L/N = 1$ in the known-concurrency sweep holds by construction (Section 7); element-level read/write mixes measure the access pattern, not the mix (Section 6); and an early TLB run requested THP but got `AnonHugePages:  0 kB`, while with THP always a "4 KiB" buffer gets huge pages unless it opts out. Both page policies are therefore set before first touch and verified (Section 10).

12.2 Limitations

- **Counters.** Memory-controller counts are system-wide; the idle background of 0.088 GiB/s reads and 0.025 GiB/s writes is small but not subtracted. Core events are user-mode, measuring thread only; generic LLC events may undercount prefetch-driven fills, and multiplexed counts are scaled.
- **Single-core traffic model not closed.** With one generator core the memory-controller counters read 0.945–0.962× the RFO-counted estimate (Section 6); the cause is an untested hypothesis.
- **The loaded-latency probe is itself load** and measures a demand read; $L = \lambda W$ for the streaming sweep assumes its latency is representative of the generators' prefetches. The known-concurrency sweep does not need that assumption.
- **One machine and configuration:** a single i9-10900F, one NUMA node, two DDR4 channels at 2933 MT/s, turbo disabled; MLC ran with prefetchers enabled (no root).

13 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

Tools used

Claude Code (Claude Opus 5.5) throughout, as recorded in `AI_USE_LOG.md`. On 2026-09-15 it produced the project scaffold (templates, Makefiles, report skeletons), which was kept. The first Project 2 benchmarks followed on 2026-10-20, after a build check of the scaffold on 2026-10-17, and the 2026-10-21 rework produced every measurement in this report. The builds used gcc (GCC) 16.2.1 20260810. The AI wrote the code. It also produced a preliminary draft of the report, which I discarded; I wrote this report myself and verified all AI output. Claude Code also ran the timed experiments, on my machine (Table 1) and with my approval, after I had made the root-only settings: performance governor, turbo off, SMT siblings offline, the

ACPI event source masked, `perf_event_paranoid`=0, 1024 reserved huge pages, and Intel MLC installed. Every number comes from those runs.

What they helped generate

The four microbenchmarks—`mem_latency` (Sattolo pointer chase with access-size, page-size and counter options), `mem_bandwidth` (multi-thread streaming with read/write/copy/triad/rwmix and access-size modes), `mem_loaded` (loaded-latency probe with streaming, read/write and known-concurrency generators) and `kernel_miss` (footprint-varying SAXPY with counters)—plus the shared helpers, the `perf_event_open` counter code, the experiment driver (`run.sh`), `validate.sh`, `derive.py`.

What was independently modified, tested, or rejected

Each item below is an AI output that was wrong and was caught by a check, a review or the data:

(1) The first latency benchmark used a *sequential* ring, so the prefetcher turned it into a ∼5 ns "latency". It was replaced by a random Sattolo cycle; the sequential curve is kept only as a contrast (Figure 6).

(2) The first huge-page experiment assumed THP backing because THP was set to `always`. A runtime `smaps_rollup` check showed `AnonHugePages:  0 kB`, so its "huge pages don't help" would have been false. The allocation was rebuilt (2 MiB-aligned mapping, `madvise` before first touch); the grant is now verified for every run, and huge pages give a 1.78× speedup (Section 10).

(3) The converse also happened: under THP `always`, a "4 KiB" buffer could silently get huge pages. `MADV_NOHUGEPAGE` is now set explicitly.

(4) An AI-suggested bandwidth test defaulted to a cache-resident buffer and reported tens of GiB/s as "DRAM bandwidth"; this is now the documented pitfall (Section 11).

(5) The read/write-mix kernel mixed 8 B elements, so every line moved in both directions; mixes are now line-granular (Section 6).

(6) The throughput–latency knee was placed by eye, and Little's Law used the zero-queue latency. Both were replaced by a concurrent loaded-latency probe and a stated knee rule that follows offered load (Section 7).

(7) A "validation" of Little's Law ($L/N \approx 1$) turned out to be an accounting identity. It was replaced by L_{mem} with the probe-measured memory latency.

(8) The scalar unit-stride kernel was quoted as the single-core peak although the strided data contradicted it (Section 5).

(9) A hand-copied table disagreed with its CSV. All tables and quoted numbers are now generated from the data (`derive.py`, `summary.csv`).

(10) Cycle counts were first converted with the in-process add-chain frequency probe, which misread the clock in about a third of runs. Upon viewing this it was flagged, and the AI's diagnostic traced it to wall time lost to an ACPI event storm, which I then masked (Section 12). Conversion now uses the fixed 2.8 GHz clock, confirmed with the cycle counter.

In a final round, the AI added Intel MLC runs (`run.sh mlc`) as the external cross-check and system-wide memory-controller counters (`uncore_imc`) that measure DRAM read/write traffic. Both became possible after I installed MLC and lowered `perf_event_paranoid`. All experiments were then re-run.

Validation

`scripts/validate.sh` asserts the physically necessary orderings (L1 < L2 < L3 < DRAM latency; L1 bandwidth > DRAM; multi-thread > single-thread; loaded > unloaded latency) and agreement of the two latency methods; its output is saved with every experiment. Hardware counters confirm the miss level behind every latency step. Intel MLC provides the external cross-check (Section 3), and the memory-controller counters check the traffic models (Sections 5 and 6). Every figure traces to a committed raw/processed file (Section A).

14 Conclusion

The hierarchy behaves as its capacities predict, and the counters show *why* each step occurs. Zero-queue latency is 1.80 ns (5.03 cycles) in L1, 5.21 ns on the L2 plateau with 4 KiB pages (L2 hit latency 4.33 ns without conflict misses), 18.6 ns in L3 and 103 ns in DRAM, of which 19.6 ns is page walking that 2 MiB pages remove. Under load, bandwidth reaches 30.6 GiB/s at only 1.37× the unloaded latency; past that point more cores buy queueing, not bandwidth, as MLC confirms. Little's Law with measured latency separates the 55.6 lines the memory system serves from up to 152 that queue; one core sustains at most 8.70 random demand misses; and counter-fitted AMAT shows random SAXPY paying 12.3 ns per DRAM miss (8.3-way overlap). Memory-controller counters confirm the RFO cost of writes and the adjacent-line cost of sparse reads. The lessons are about measurement discipline: out-size the LLC (Section 11), randomize latency chases, measure loaded latency, mix reads and writes per line, verify huge pages (Section 10), and measure the clock (Section 12).

References

[1] Intel Corporation, *Intel memory latency checker*, Version 3.13 on the test host; used as an independent cross-check (random-access mode, prefetchers left enabled).

[2] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961. DOI: 10.1287/opre.9.3.383

[3] *Perf: Linux profiling with performance counters*. [Online]. Available: https://perf.wiki.kernel.org/

A Reproducibility

Code, scripts, raw and processed data, and this report live in one repository, in this project's `code/` and `latex/` directories (shared files in `common/`), at commit `d1e7b4b7fc69`.

```
gh repo clone SKempiak/ACS-Code ACS && cd ACS
git checkout d1e7b4b7fc69
make tools                                          # common/code/build/tools/summarize
# build mem_latency mem_bandwidth mem_loaded kernel_miss; run the validate.sh sanity checks
make -C Cache_and_Memory_Performance_Profiling/code
make -C Cache_and_Memory_Performance_Profiling/code test
common/code/scripts/collect_env.sh --tex > common/latex/system_info.tex
Cache_and_Memory_Performance_Profiling/code/scripts/run.sh zero_queue pattern rw_mix intensity \
    working_set cache_miss tlb pitfall mlc clock_check   # or: run.sh all (CPU_PIN=2 by default)
common/code/scripts/sync_data.sh Cache_and_Memory_Performance_Profiling
make memory
```

Table 5: Raw data behind each figure and table (directories under `Cache_and_Memory_Performance_Profiling/code/data/raw/`).

Figure / table	`run.sh` experiment	Processed CSV	Raw run directory
Table 2	`zero_queue`	`zero_queue`, `zero_queue_table` (L2 2 MiB value: `working_set`)	`zero_queue/20261021-202254`
Figures 1 and 2	`pattern`	`pattern_lat`, `pattern_bw`	`pattern/20261021-202254`
Figure 3	`rw_mix`	`rw_mix`, `rw_mix_ref`	`rw_mix/20261021-202254`
Figures 4 and 5, Table 3	`intensity`	`intensity_loaded`, `intensity_mlp`, `littles_law`, `knee`	`intensity/20261021-202254`
Figure 6	`working_set`	`working_set`	`working_set/20261021-202254`
Figure 7	`cache_miss`	`cache_miss_ctr`, `amat_fit`	`cache_miss/20261021-202254`
Figure 8	`tlb`	`tlb_chase`	`tlb/20261021-202254`
Table 4	`tlb`	`tlb_saxpy`, `tlb_saxpy_table`	`tlb/20261021-202254`
Figure 9	`pitfall`	`pitfall`	`pitfall/20261021-202254`
Table 2 and Figure 4, Section 6 (MLC)	`mlc`	`mlc_idle`, `mlc_loaded`, `mlc_bw`	`mlc/20261021-202254`
Section 2 (clock, idle DRAM traffic)	`clock_check`	— (`summary.csv`)	`clock_check/20261021-202254`
Section 12 (ACPI storm)	`clock_check`	— (`summary.csv`)	`clock_check/20261021-194648`

Each experiment directory holds the raw rows (one per repetition, with full configuration metadata), `env.txt`, `build.txt`, `commands.txt` and `validate.txt` (Section 2), except that the MLC directory (Intel's binary; raw text output plus `mlc_version.txt`, parsed by `scripts/mlc_to_csv.py`) has no `build.txt` or `validate.txt`, and the earlier `clock_check/20261021-194648` run, kept as evidence for Section 12, has no `validate.txt`. All other directories come from a single `run.sh all` invocation (2026-10-21, 20:22–20:39) of code identical to commit `b00db66` (Section 2).